Unit 3 - Structured Query Language (SQL)

Database Management (DI03016041) - GTU Diploma IT Sem 3

25% weightage | 14 hours (BIGGEST unit - practise the queries by writing them)

How to use these notes: every topic is written in a descriptive, copy-ready way. **3 marks** = definition + 2-3 points. **4 marks** = add example or table. **7 marks** = write everything with diagram / example. Lines marked ★ show where the topic was asked in a GTU paper (Winter 2025 / Summer 2026). SQL examples use Oracle syntax.

GTU questions already asked from this unit (do these first)

- Asked in S2026 Q3(b)OR [4M: explain DDL commands] + W2025 Q3(a) [3M: define DDL]
- Asked in S2026 Q3(b) [4M: explain DML commands] + W2025 Q3(a)OR [3M: define DML]
- Asked in S2026 Q4(a)OR [3M: example of GROUP BY clause]
- DCL asked in BOTH papers: W2025 Q3(a) [Grant] + Q3(a)OR [Revoke] + S2026 Q3(a)OR [3M: GRANT and REVOKE]
- Short definitions asked: W2025 Q3(a) [DDL, Grant, SQL View] + Q3(a)OR [DML, Revoke, Inner Join]
- Asked in W2025 Q3(a) [3M: define SQL View]
- Aggregate functions asked in BOTH papers: W2025 Q3(b)OR [4M: MIN, COUNT] + S2026 Q3(a) [3M: any three with example]
- Asked in S2026 Q4(a) [3M: any three numeric functions with example]
- Asked in W2025 Q3(b) [4M: LENGTH and MONTHS_BETWEEN]
- Joins asked in BOTH papers: W2025 Q3(a)OR [3M: define Inner Join] + S2026 Q4(b) [4M: INNER JOIN with example]
- Asked in S2026 Q3(c)OR [7M: explain SQL constraints] + CREATE TABLE questions W2025 Q4(b), Q4(b)OR, S2026 Q5(a)
- W2025 Q4(b) [4M: create STUDENT table with primary key and branch check]
- S2026 Q5(a) [3M: create STUDENT table, stu_name NOT NULL, branch check]
- W2025 Q4(b)OR [4M: create PRODUCT table, prod_name not null, price > 0]
- W2025 Q3(c) [7M: SQL queries on BOOK_Master]
- W2025 Q3(c)OR [7M: SQL queries on EMPLOYEE_Master]
- S2026 Q3(c) [7M: SQL statements on Student table]

3.1 Introduction, command groups, SQL data types

SQL (Structured Query Language) is the standard language used to create, manipulate, query and control relational databases (Oracle, MySQL, SQL Server). Commands are grouped by purpose:

Group	Full form	Purpose	Commands
DDL	Data Definition Language	Defines / changes the **structure** of database objects	CREATE, ALTER, TRUNCATE, DROP, RENAME
DML	Data Manipulation Language	Changes the **data** inside tables	INSERT, UPDATE, DELETE
DQL	Data Query Language	**Retrieves** data	SELECT
DCL	Data Control Language	Controls **access / privileges**	GRANT, REVOKE
TCL	Transaction Control Language	Controls transactions	COMMIT, ROLLBACK, SAVEPOINT

All examples in these notes use **Oracle SQL** (the syllabus lab uses Oracle). Every statement ends with a semicolon (;). SQL keywords are not case-sensitive, but text values inside quotes are.

SQL data types (Oracle)

Data type	Use	Example
CHAR(n)	Fixed-length text (max 2000)	CHAR(10)
VARCHAR2(n)	Variable-length text (max 4000)	VARCHAR2(30) for names
NUMBER(p, s)	Numbers; p = total digits, s = digits after decimal point	NUMBER(8,2) = 123456.78
DATE	Date and time	'15-JAN-2025'
LONG / CLOB	Large text	Description
BLOB	Binary data (image, file)	Photo

3.1.2 DDL commands: create, alter, truncate, drop

★ Asked in S2026 Q3(b)OR [4M: explain DDL commands] + W2025 Q3(a) [3M: define DDL]

DDL (Data Definition Language) commands define or change the **structure (schema)** of database objects such as tables. DDL commands are **auto-committed** (cannot be rolled back).

Command	Purpose	Syntax and example
CREATE	Creates a new table (or other object)	CREATE TABLE student (roll_no NUMBER(5), name VARCHAR2(30), city VARCHAR2(20));
ALTER	Changes structure of an existing table: add / modify / drop column	ALTER TABLE student ADD phone NUMBER(10); ALTER TABLE student MODIFY name VARCHAR2(50); ALTER TABLE student DROP COLUMN city;
TRUNCATE	Deletes **all rows** quickly but keeps the table structure; cannot be rolled back	TRUNCATE TABLE student;
DROP	Deletes the **table with its data and structure** permanently	DROP TABLE student;
RENAME	Changes the name of a table	RENAME student TO student_master;

Describe shows the structure of a table: DESC `student;`

TRUNCATE	DELETE	DROP
DDL; removes all rows	DML; removes selected (or all) rows with WHERE	DDL; removes table itself
Cannot rollback, no WHERE	Can rollback, WHERE allowed	Cannot rollback
Structure remains	Structure remains	Structure and data removed

3.1.3 DML commands: insert, update, delete

★ Asked in S2026 Q3(b) [4M: explain DML commands] + W2025 Q3(a)OR [3M: define DML]

DML (Data Manipulation Language) commands are used to **insert, modify and delete the data** stored in tables. They can be rolled back (until COMMIT).

1. INSERT - adds new rows

```
INSERT INTO student (roll_no, name, city) VALUES (101, 'Ravi', 'Surat');
INSERT INTO student VALUES (102, 'Meera', 'Rajkot');      -- values for all columns
```

2. UPDATE - changes existing data

```
UPDATE student SET city = 'Vadodara' WHERE roll_no = 101;     -- only one row
UPDATE student SET fees = fees + 500;                           -- all rows (no WHERE)
```

3. DELETE - removes rows

```
DELETE FROM student WHERE roll_no = 102;     -- one row
DELETE FROM student;                             -- all rows, table remains
```

Always use **WHERE** with UPDATE and DELETE, otherwise all rows are affected. SELECT is the DQL command to read data.

3.1.4 DQL (select) and miscellaneous: describe, distinct, order by, group by, having

★ Asked in S2026 Q4(a)OR [3M: example of GROUP BY clause]

SELECT - Data Query Language (DQL)

```
SELECT * FROM student;                                  -- all columns, all rows
SELECT name, city FROM student WHERE city = 'Surat';   -- selected columns and rows
SELECT DISTINCT city FROM student;                       -- removes duplicates
SELECT * FROM student ORDER BY name;                     -- ascending (default)
SELECT * FROM student ORDER BY fees DESC;                -- descending
```

General form: `SELECT columns FROM table [WHERE condition] [GROUP BY column] [HAVING condition] [ORDER BY column];`

Other miscellaneous commands

Clause / command	Purpose	Example
DESCRIBE (DESC)	Shows structure of a table (columns, types, null)	DESC student;
DISTINCT	Removes duplicate rows from the result	SELECT DISTINCT branch FROM student;
ORDER BY	Sorts result: ASC (default) or DESC	SELECT * FROM book ORDER BY price DESC;
GROUP BY	Groups rows having same value so aggregate functions work per group	SELECT dept, SUM(salary) FROM emp GROUP BY dept;
HAVING	Filters **groups** (WHERE filters rows, HAVING filters groups)	SELECT dept, COUNT(*) FROM emp GROUP BY dept HAVING COUNT(*) > 2;

GROUP BY groups rows that have the same value in the given column(s) into summary rows, so that an **aggregate function** is applied to each group.

Table **EMP**:

emp_id	emp_name	dept	salary
1	Ravi	IT	60000
2	Meera	HR	40000
3	Amit	IT	50000
4	Neha	HR	45000
5	Sunil	Sales	35000

```
SELECT dept, COUNT(*) AS total_emp, SUM(salary) AS total_salary
FROM emp
GROUP BY dept
HAVING SUM(salary) > 40000;
```

dept	total_emp	total_salary
IT	2	110000
HR	2	85000

Sales (35000) is removed by HAVING. Every non-aggregate column in SELECT must appear in GROUP BY.

3.1.5 Privilege commands: grant and revoke (and TCL)

★ DCL asked in BOTH papers: W2025 Q3(a) [Grant] + Q3(a)OR [Revoke] + S2026 Q3(a)OR [3M: GRANT and REVOKE]

DCL (Data Control Language) commands control **who can access which database objects** - used by the DBA / owner for security.

- **GRANT** - gives privileges (SELECT, INSERT, UPDATE, DELETE, ALL) on an object to a user or role.

```
GRANT SELECT, INSERT ON student TO ravi;
GRANT ALL ON student TO meera WITH GRANT OPTION;   -- meera can pass the privilege on to others
```

- **REVOKE** - takes back privileges previously given.

```
REVOKE INSERT ON student FROM ravi;
REVOKE ALL ON student FROM meera;
```

User management (lab): `CREATE USER ravi IDENTIFIED BY pass123;` `GRANT CREATE SESSION TO ravi;` `DROP USER ravi;`

TCL (for completeness): `COMMIT;` saves changes permanently, `ROLLBACK;` undoes uncommitted changes, `SAVEPOINT sp1;` marks a point to roll back to.

Short definitions: DDL, DML, Grant, Revoke, View, Inner join

★ Short definitions asked: W2025 Q3(a) [DDL, Grant, SQL View] + Q3(a)OR [DML, Revoke, Inner Join]

Term	Definition (3-mark style)
DDL	Data Definition Language - commands that define or change the structure of database objects: CREATE, ALTER, TRUNCATE, DROP, RENAME.
DML	Data Manipulation Language - commands that insert, change and delete the data in tables: INSERT, UPDATE, DELETE.
GRANT	DCL command that gives privileges on a database object to a user. Example: GRANT SELECT, INSERT ON student TO ravi;
REVOKE	DCL command that takes back privileges given earlier. Example: REVOKE INSERT ON student FROM ravi;
SQL View	A virtual table created from a SELECT query; it stores no data of its own. Example: CREATE VIEW it_students AS SELECT * FROM student WHERE branch='IT';
Inner Join	Join that returns only the rows having matching values in both tables. Example: SELECT * FROM student s INNER JOIN department d ON s.dept_id = d.dept_id;

3.2 SQL views

★ Asked in W2025 Q3(a) [3M: define SQL View]

A **view** is a **virtual table** based on the result of a SELECT query. It does not store data itself - it shows data of one or more base tables.

```
CREATE VIEW it_students AS
SELECT roll_no, name FROM student WHERE branch = 'IT';

SELECT * FROM it_students;      -- use like a table
DROP VIEW it_students;
```

- **Advantages:** security (hide columns / rows), simplifies complex queries, logical data independence.
- Changes in the base table are reflected in the view automatically.

3.3 SQL functions: aggregate

★ Aggregate functions asked in BOTH papers: W2025 Q3(b)OR [4M: MIN, COUNT] + S2026 Q3(a) [3M: any three with example]

Aggregate (group) functions work on a set of rows and return **one single value**. They ignore NULL values (except COUNT(*)).

Table **EMP(emp_id, emp_name, dept, salary)**: salaries 60000, 40000, 50000, 45000, 35000 (5 rows).

Function	Purpose	Example query	Result
MIN()	Smallest value	SELECT MIN(salary) FROM emp;	35000
MAX()	Largest value	SELECT MAX(salary) FROM emp;	60000
COUNT()	Number of rows. COUNT(*) counts all rows; COUNT(col) counts non-NULL values	SELECT COUNT(*) FROM emp;	5
SUM()	Total of a numeric column	SELECT SUM(salary) FROM emp;	230000
AVG()	Average value	SELECT AVG(salary) FROM emp;	46000

3.3 SQL functions: numeric

★ Asked in S2026 Q4(a) [3M: any three numeric functions with example]

Numeric (single-row) functions take numbers and return numbers. Use the dummy table **DUAL** in Oracle to test them.

Function	Purpose	Example	Result
ABS(n)	Absolute value	SELECT ABS(-15) FROM DUAL;	15
CEIL(n)	Smallest integer ≥ n	SELECT CEIL(4.2) FROM DUAL;	5
FLOOR(n)	Largest integer ≤ n	SELECT FLOOR(4.8) FROM DUAL;	4
ROUND(n, d)	Rounds to d decimal places	SELECT ROUND(45.678, 2) FROM DUAL;	45.68
TRUNC(n, d)	Cuts off after d decimals (no rounding)	SELECT TRUNC(45.678, 1) FROM DUAL;	45.6
MOD(m, n)	Remainder of m ÷ n	SELECT MOD(10, 3) FROM DUAL;	1
POWER(m, n)	m raised to power n	SELECT POWER(2, 3) FROM DUAL;	8
SQRT(n)	Square root	SELECT SQRT(16) FROM DUAL;	4

3.3 SQL functions: character and date

★ Asked in W2025 Q3(b) [4M: LENGTH and MONTHS_BETWEEN]

Character (string) functions

Function	Purpose	Example	Result
LENGTH(s)	**Number of characters** in a string (including spaces)	SELECT LENGTH('DATABASE') FROM DUAL;	8
UPPER(s) / LOWER(s)	Converts case	SELECT UPPER('gtu') FROM DUAL;	GTU
INITCAP(s)	First letter of each word capital	SELECT INITCAP('jay patel') FROM DUAL;	Jay Patel
SUBSTR(s, start, n)	Part of a string	SELECT SUBSTR('DATABASE', 1, 4) FROM DUAL;	DATA
INSTR(s, x)	Position of x in s	SELECT INSTR('DATABASE', 'B') FROM DUAL;	5
CONCAT(a, b) or ||	Joins strings	SELECT 'DB' || 'MS' FROM DUAL;	DBMS
REPLACE(s, a, b)	Replaces text	SELECT REPLACE('cat', 'c', 'b') FROM DUAL;	bat
LPAD / RPAD / TRIM	Pad or remove spaces	SELECT LPAD('5', 3, '0') FROM DUAL;	005

Date functions

Function	Purpose	Example	Result
MONTHS_BETWEEN(d1, d2)	**Number of months between two dates** (d1 later than d2 gives a positive number; can be fractional)	SELECT MONTHS_BETWEEN(TO_DATE('01-07-2025','DD-MM-YYYY'), TO_DATE('01-01-2025','DD-MM-YYYY')) FROM DUAL;	6
SYSDATE	Current system date and time	SELECT SYSDATE FROM DUAL;	today's date
ADD_MONTHS(d, n)	Adds n months to a date	SELECT ADD_MONTHS('15-JAN-2025', 2) FROM DUAL;	15-MAR-2025
LAST_DAY(d)	Last day of the month	SELECT LAST_DAY('10-FEB-2025') FROM DUAL;	28-FEB-2025
NEXT_DAY(d, 'day')	Date of the next given weekday	SELECT NEXT_DAY(SYSDATE, 'MONDAY') FROM DUAL;	next Monday

3.4 SQL operators

Type	Operators	Example
Arithmetic	+ - * /	SELECT salary * 12 FROM emp;
Comparison	= <> (or !=) > < >= <=	WHERE salary >= 50000
Logical	AND, OR, NOT	WHERE dept = 'IT' AND salary > 40000
BETWEEN	Value in a range (inclusive)	WHERE salary BETWEEN 40000 AND 60000
IN / NOT IN	Value in a list	WHERE city IN ('Surat', 'Rajkot')
LIKE	Pattern match: % = any characters, _ = one character	WHERE name LIKE 'A%' (starts with A); LIKE '_a%' (second letter a)
IS NULL / IS NOT NULL	Tests for NULL	WHERE phone IS NULL

3.5 Set operators: union, union all, intersect, minus

Set operators combine the results of two SELECT queries (both must have the same number of columns with compatible data types). Tables: **ACCOUNT(acc_no)** = A1, A2, A3, A4 and **LOAN(acc_no)** = A1, A3, A5.

Operator	Meaning	Example	Result
UNION	All rows of both queries, **duplicates removed**	SELECT acc_no FROM account UNION SELECT acc_no FROM loan;	A1, A2, A3, A4, A5
UNION ALL	All rows of both queries, **duplicates kept**	... UNION ALL ...	A1, A2, A3, A4, A1, A3, A5
INTERSECT	Only rows **common** to both	... INTERSECT ...	A1, A3
MINUS	Rows in the first query that are **not in the second**	SELECT acc_no FROM account MINUS SELECT acc_no FROM loan;	A2, A4

3.6 Joins

★ Joins asked in BOTH papers: W2025 Q3(a)OR [3M: define Inner Join] + S2026 Q4(b) [4M: INNER JOIN with example]

A **join** combines rows of two or more tables **based on a related column** (usually primary key = foreign key).

INNER JOIN returns **only the rows that have matching values in both tables**; rows without a match are left out.

STUDENT(roll_no, name, dept_id) and **DEPARTMENT**(dept_id, dept_name):

STUDENT
1 Ravi 10
2 Meera 20
3 Amit 30

DEPARTMENT
10 IT
20 Computer
40 Civil

```
SELECT s.roll_no, s.name, d.dept_name
FROM student s INNER JOIN department d
ON s.dept_id = d.dept_id;
```

roll_no	name	dept_name
1	Ravi	IT
2	Meera	Computer

Amit (dept 30) and Civil (40) have no match, so they do not appear.

Other types of joins

Join	Result
LEFT OUTER JOIN	All rows of left table + matching rows of right (NULL where no match)
RIGHT OUTER JOIN	All rows of right table + matching rows of left
FULL OUTER JOIN	All rows of both tables, NULL where no match
CROSS JOIN	Every row of one table with every row of the other (Cartesian product)
SELF JOIN	A table joined with itself (e.g. employee and manager in the same table)

3.7 SQL constraints

★ Asked in S2026 Q3(c)OR [7M: explain SQL constraints] + CREATE TABLE questions W2025 Q4(b), Q4(b)OR, S2026 Q5(a)

Constraints are **rules applied on columns of a table** to keep the data **accurate, valid and consistent** (data integrity). The database rejects any INSERT / UPDATE that breaks a rule.

Need of constraints

- Prevent invalid data (negative price, missing name).
- Avoid duplicate rows and keep rows identifiable.
- Maintain correct links between tables (no order for non-existing customer).
- Enforce business rules in the database itself, not only in programs.

Types of constraints

Integrity type	Constraint	Meaning	Example
Domain integrity	**NOT NULL**	Column cannot be empty	name VARCHAR2(30) NOT NULL
	CHECK	Value must satisfy a condition	price NUMBER CHECK (price > 0)
	DEFAULT	Value used when none is given	city VARCHAR2(20) DEFAULT 'Surat'
Entity integrity	**UNIQUE**	No duplicate values (NULL allowed)	email VARCHAR2(50) UNIQUE
	PRIMARY KEY	Unique + NOT NULL; identifies each row; one per table	roll_no NUMBER(5) PRIMARY KEY
Referential integrity	**FOREIGN KEY** (REFERENCES)	Value must exist in the primary / reference key of another table	dept_id NUMBER REFERENCES department(dept_id)

Example with all constraints

```
CREATE TABLE department (
  dept_id   NUMBER(3) PRIMARY KEY,
  dept_name VARCHAR2(20) NOT NULL UNIQUE
);

CREATE TABLE student (
  stu_id   NUMBER(5) PRIMARY KEY,
  stu_name VARCHAR2(30) NOT NULL,
  email    VARCHAR2(40) UNIQUE,
  fees     NUMBER(8,2) CHECK (fees >= 0),
  city     VARCHAR2(20) DEFAULT 'Surat',
  dept_id  NUMBER(3),
  CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

Constraints can be added later: `ALTER TABLE student ADD CONSTRAINT chk_f CHECK (fees >= 0);` and dropped with `ALTER TABLE student DROP CONSTRAINT chk_f;`

CREATE TABLE questions asked: STUDENT (W2025)

★ W2025 Q4(b) [4M: create STUDENT table with primary key and branch check]

```
CREATE TABLE STUDENT (
  stu_id      NUMBER(5) PRIMARY KEY,
  stu_name    VARCHAR2(30),
  birthdate   DATE,
  City        VARCHAR2(20),
  contact_no  NUMBER(10),
  Branch_name VARCHAR2(15)
              CHECK (Branch_name IN ('IT', 'Computer', 'Electrical', 'Civil'))
);
```

(i) PRIMARY KEY on stu_id makes it unique and not null. (ii) CHECK ... IN (...) allows only the four branch names.

CREATE TABLE questions asked: STUDENT (S2026)

★ S2026 Q5(a) [3M: create STUDENT table, stu_name NOT NULL, branch check]

```
CREATE TABLE STUDENT (
  stu_id      NUMBER(5) PRIMARY KEY,
  stu_name    VARCHAR2(30) NOT NULL,
  Address     VARCHAR2(50),
  City        VARCHAR2(20),
  contact_no  NUMBER(10),
  Branch_name VARCHAR2(15)
              CHECK (Branch_name IN ('IT', 'Computer', 'Electrical', 'Mechanical'))
);
```

(i) NOT NULL stops empty names. (ii) CHECK allows only 'IT', 'Computer', 'Electrical', 'Mechanical'.

CREATE TABLE questions asked: PRODUCT

★ W2025 Q4(b)OR [4M: create PRODUCT table, prod_name not null, price > 0]

```
CREATE TABLE PRODUCT (
  prod_id    NUMBER(5) PRIMARY KEY,
  prod_name  VARCHAR2(30) NOT NULL,
  price      NUMBER(8,2) CHECK (price > 0),
  mfg_date   DATE
);
```

(i) NOT NULL on prod_name. (ii) CHECK (price > 0) rejects zero or negative prices.

Solved SQL queries: BOOK_Master

★ W2025 Q3(c) [7M: SQL queries on BOOK_Master]

BOOK_Master(book_id, title, author, price, publisher)

1. Insert a new book

```
INSERT INTO BOOK_Master (book_id, title, author, price, publisher)
VALUES (101, 'Database System Concepts', 'Korth', 750, 'Pearson');
```

2. Display all books published by 'Pearson'

```
SELECT * FROM BOOK_Master WHERE publisher = 'Pearson';
```

3. Books whose title begins with 'D'

```
SELECT * FROM BOOK_Master WHERE title LIKE 'D%';
```

4. Add a new column edition

```
ALTER TABLE BOOK_Master ADD edition NUMBER(2);
```

5. Count the number of rows

```
SELECT COUNT(*) FROM BOOK_Master;
```

6. All details in descending order of price

```
SELECT * FROM BOOK_Master ORDER BY price DESC;
```

7. Destroy the table along with its data

```
DROP TABLE BOOK_Master;
```

Solved SQL queries: EMPLOYEE_Master

★ W2025 Q3(c)OR [7M: SQL queries on EMPLOYEE_Master]

EMPLOYEE_Master(emp_id, emp_name, dept, salary, hire_date)

1. Create table

```
CREATE TABLE EMPLOYEE_Master (
  emp_id    NUMBER(5) PRIMARY KEY,
  emp_name  VARCHAR2(30),
  dept      VARCHAR2(20),
  salary    NUMBER(10,2),
  hire_date DATE
);
```

2. Insert one record

```
INSERT INTO EMPLOYEE_Master
VALUES (1, 'Ravi Patel', 'IT', 60000, TO_DATE('15-01-2023', 'DD-MM-YYYY'));
```

3. Employees having salary greater than 50,000

```
SELECT * FROM EMPLOYEE_Master WHERE salary > 50000;
```

4. Delete employees of 'HR' department

```
DELETE FROM EMPLOYEE_Master WHERE dept = 'HR';
```

5. Increase salary by 10% for 'IT' department

```
UPDATE EMPLOYEE_Master SET salary = salary + salary * 10 / 100 WHERE dept = 'IT';
-- or: SET salary = salary * 1.10
```

6. Add new field email

```
ALTER TABLE EMPLOYEE_Master ADD email VARCHAR2(50);
```

7. Total salary of all employees of 'IT' department

```
SELECT SUM(salary) FROM EMPLOYEE_Master WHERE dept = 'IT';
```

Solved SQL queries: Student

★ S2026 Q3(c) [7M: SQL statements on Student table]

Student(Enno, name, branch, sem, clgname, bdate)

1. Create table Student

```
CREATE TABLE Student (
  Enno    NUMBER(12) PRIMARY KEY,
  name    VARCHAR2(30),
  branch  VARCHAR2(20),
  sem     NUMBER(1),
  clgname VARCHAR2(50),
  bdate   DATE
);
```

2. Add a column mobno

```
ALTER TABLE Student ADD mobno NUMBER(10);
```

3. Insert one record

```
INSERT INTO Student (Enno, name, branch, sem, clgname, bdate)
VALUES (230001, 'Ankit', 'IT', 3, 'Govt Polytechnic', TO_DATE('12-05-2006', 'DD-MM-YYYY'));
```

4. Students enrolled in 'IT' branch

```
SELECT * FROM Student WHERE branch = 'IT';
```

5. All information where name begins with 'a'

```
SELECT * FROM Student WHERE name LIKE 'a%';
-- ignore case: WHERE LOWER(name) LIKE 'a%'
```

6. Count the number of rows

```
SELECT COUNT(*) FROM Student;
```

7. Delete all records

```
DELETE FROM Student;      -- or TRUNCATE TABLE Student; (faster, cannot be rolled back)
```

Last-minute revision sheet - Unit 3

- DDL: CREATE ALTER TRUNCATE DROP RENAME | DML: INSERT UPDATE DELETE | DQL: SELECT | DCL: GRANT REVOKE | TCL: COMMIT ROLLBACK SAVEPOINT.
- SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ...
- LIKE 'D%' = starts with D. COUNT(*) counts rows. ORDER BY price DESC = descending.
- ALTER TABLE t ADD col type; MODIFY col type; DROP COLUMN col. DROP TABLE removes table; TRUNCATE removes only rows.
- Functions: LENGTH, MONTHS_BETWEEN, MIN, MAX, COUNT, SUM, AVG, ROUND, CEIL, FLOOR, MOD, POWER.
- Constraints: NOT NULL, CHECK, UNIQUE, PRIMARY KEY, FOREIGN KEY (+DEFAULT).
- CHECK (col IN ('A','B')) limits values; CHECK (price > 0) limits range.
- UNION removes duplicates; UNION ALL keeps; INTERSECT = common; MINUS = first not in second.
- INNER JOIN = only matching rows.